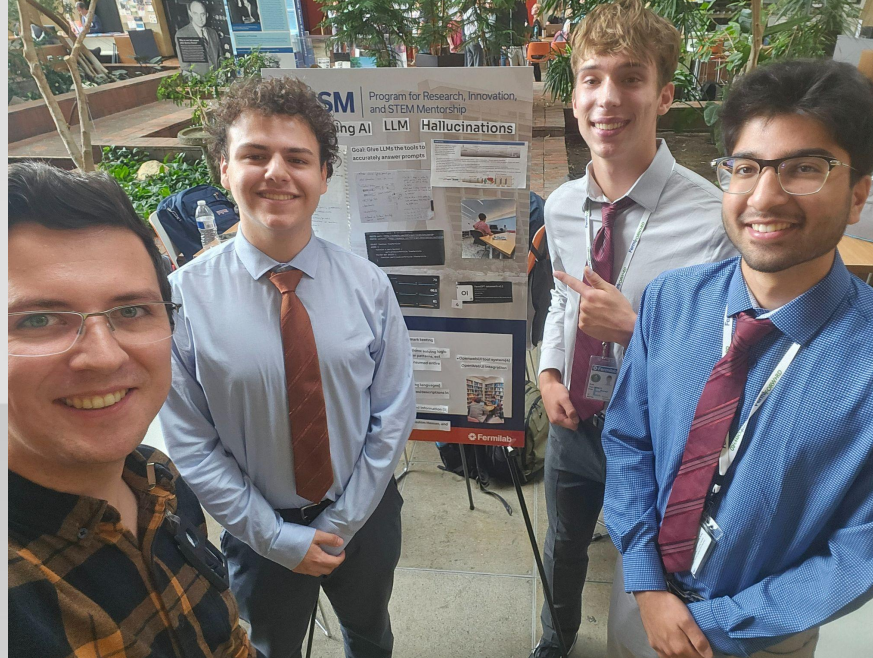




August 7, 2026

Limiting Hallucinations in AI LLMs

Hashim, Evan, Rhubyn

The best AI intern group you'll ever meet

Special thanks to our mentor,
Javier Contreras

And for extra support: Matteo Pionton
Modiano and David Espinoza

This manuscript has been authored by FermiForward Discovery Group, LLC under Contract No. 89243024CSC000002 with the U.S. Department of Energy, Office of Science, Office of High Energy Physics. This work was supported in part by the U.S. Department of Energy, Office of Science, Office of Workforce Development for Teachers and Scientists (WDTS) under the Pathway Summer Schools Program.



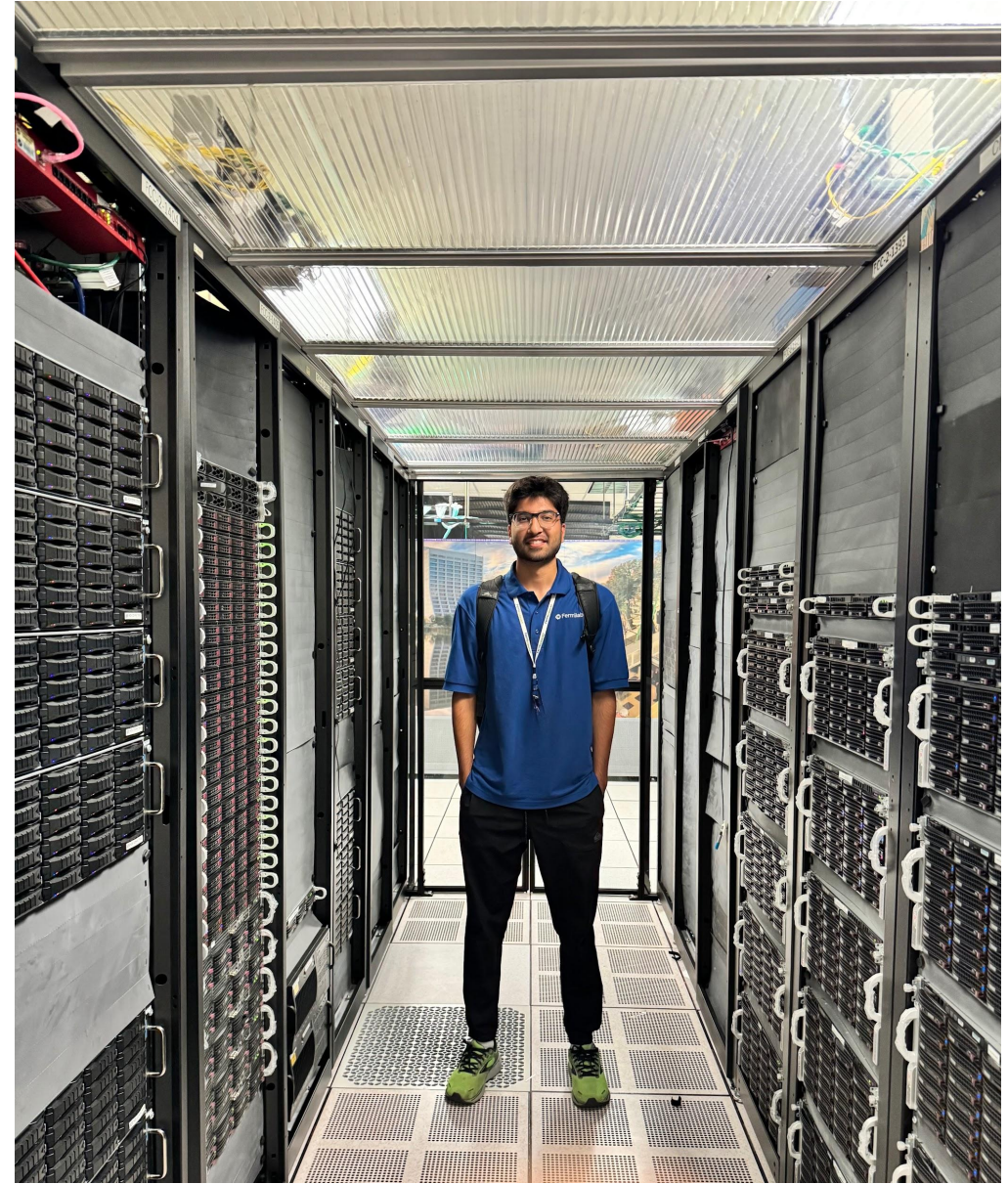
U.S. DEPARTMENT
of **ENERGY**

Fermi National Accelerator Laboratory is managed by
FermiForward for the U.S. Department of Energy Office of Science

Meet the Team!

Hashim Hassan

- Rising Freshman at the University of Illinois at Urbana-Champaign
- Studying Mechanical Engineering
- Soccer + Track and Field
- Love cars, favorites are Ferraris and Aston Martins

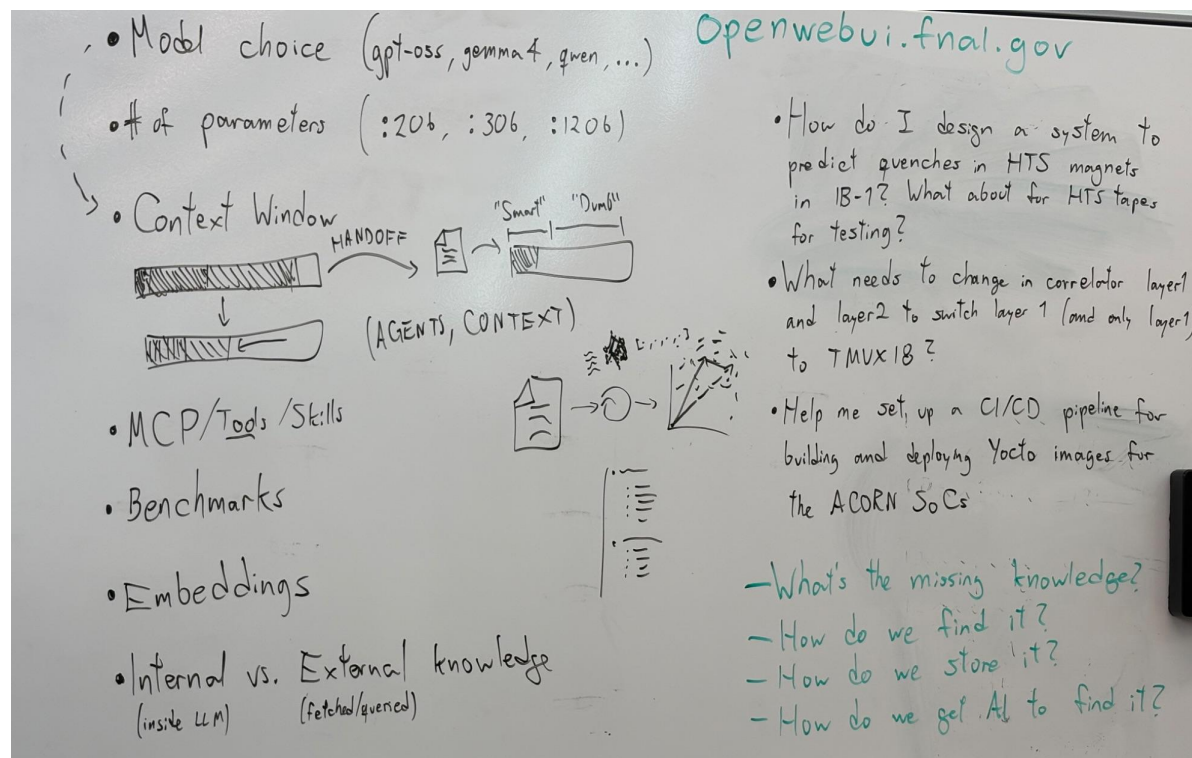


Hashim at the Feynman Computing Center

Our Goal

Our project goal was to train an AI model to be able to return accurate data regarding FermiLab detectors and answer questions about said detectors and their software.

To achieve this goal we would need to significantly reduce the number of hallucinations an AI Model creates



Openwebui.fnl.gov

- Model choice (gpt-oss, gemma4, qwen, ...)
- # of parameters (:20b, :30b, :120b)
- Context Window
 - Diagram showing a "Context Window" bar with a "HANDOFF" arrow pointing to a box labeled "(AGENTS, CONTEXT)". This box then points to a bar divided into "Smart" and "Dumb" sections.
- MCP/Tods/Skills
- Benchmarks
- Embeddings
- Internal vs. External knowledge
 - (inside LLM) vs. (fetchal/queried)

Additional notes on the right side of the whiteboard:

- How do I design a system to predict quenches in HTS magnets in IB-1? What about for HTS tapes for testing?
- What needs to change in correlator layer1 and layer2 to switch layer 1 (and only layer1) to TMUX18?
- Help me set up a CI/CD pipeline for building and deploying Yocto images for the ACORN SoCs
- What's the missing knowledge?
- How do we find it?
- How do we store it?
- How do we get AI to find it?



What Are Hallucinations? And why do LLMs have them?

- Hallucinations are the main problem plaguing Large Language Models(LLMs)
- A Hallucination is when the AI fabricates details in its response
- LLMs are trained to predict and create fluent humanlike sentences not exactly give correct answers—this leads to hallucinations
- The easiest way to get around hallucinations is to expand the knowledge base included in your prompt





Previous attempts at combating Hallucinations

Previous attempts at solving this problem have been unsuccessful, such as:

- Just make the model bigger!
- Prompt the model to “not make mistakes”
- “Cite your evidence”



ummmmm 5

yo chatgpt whats 2+2?? DONT MAKE MISTAKES >:(



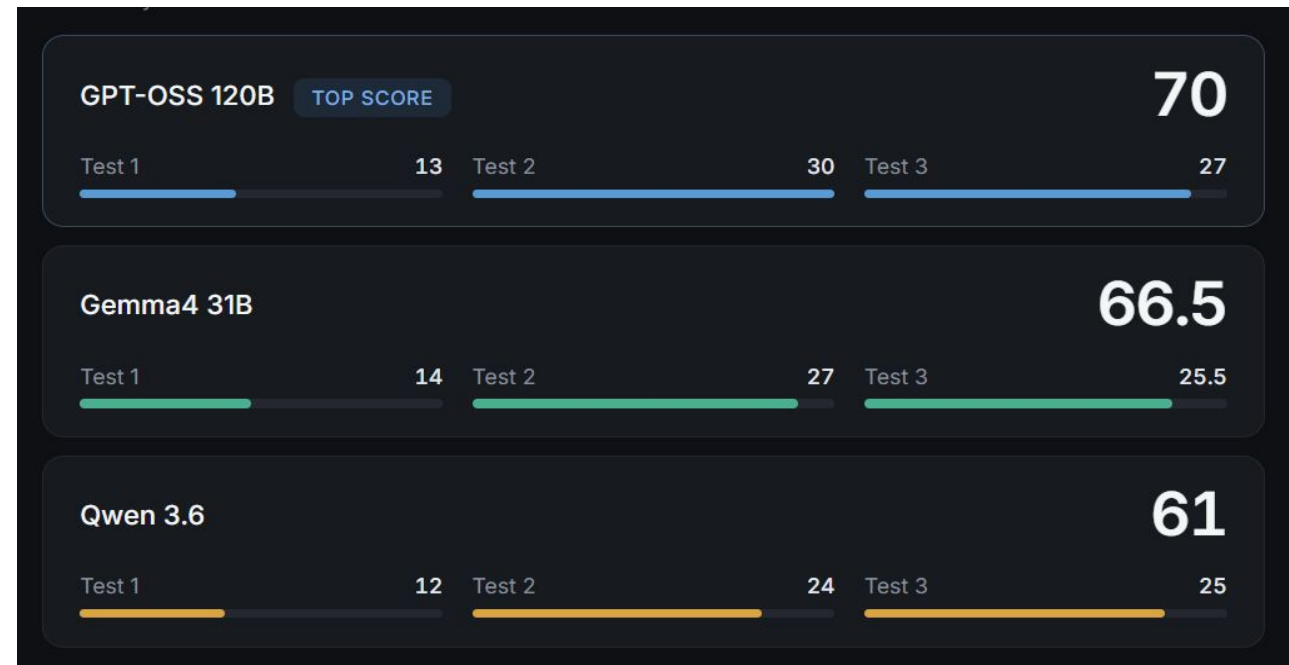
Benchmark Testing

- Models are often trained on publicly available benchmarks, so we wrote our own questions for a benchmark test.
- We gave the AI three tasks: solving a logic puzzle, finding a hidden pattern, and catching errors in fake documents.

A key challenge:

GPT-OSS

consumed its entire token budget by reasoning internally and returning zero output until limits were raised.





How we tried to combat hallucinations

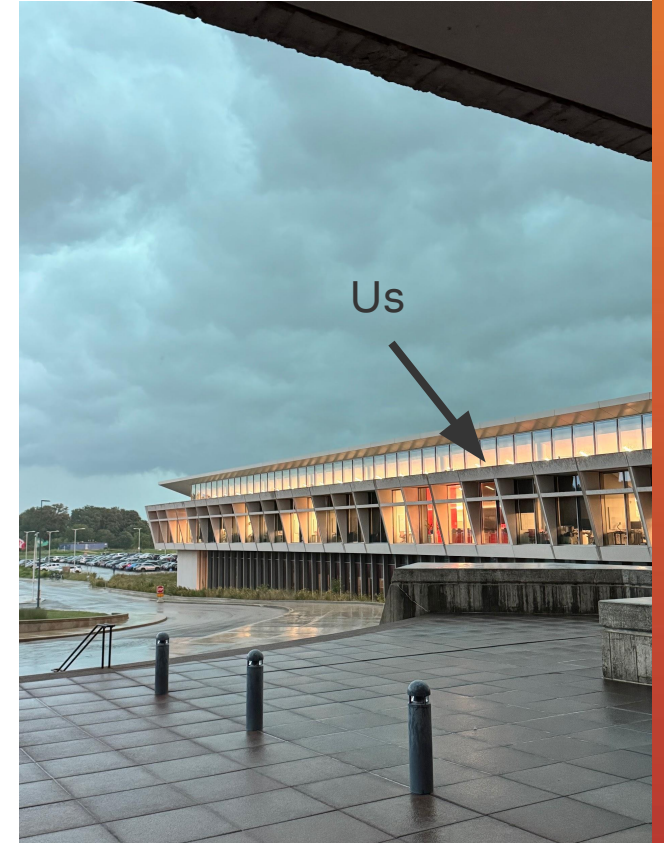
- To improve the way LLMs decide what information to analyse, we organized the data into vocabularies and descriptions
- We created vocabularies/descriptions in OML for MI-8 Targets/Detectors and the CMS Detector

Using queries we were able to pull the necessary information

These queries were created by a trained AI

We used many tools to accomplish this such as:

OML Rosetta and Open WebUI



Our home (2pm-4pm)

What is SPARQL? Why do we use it?

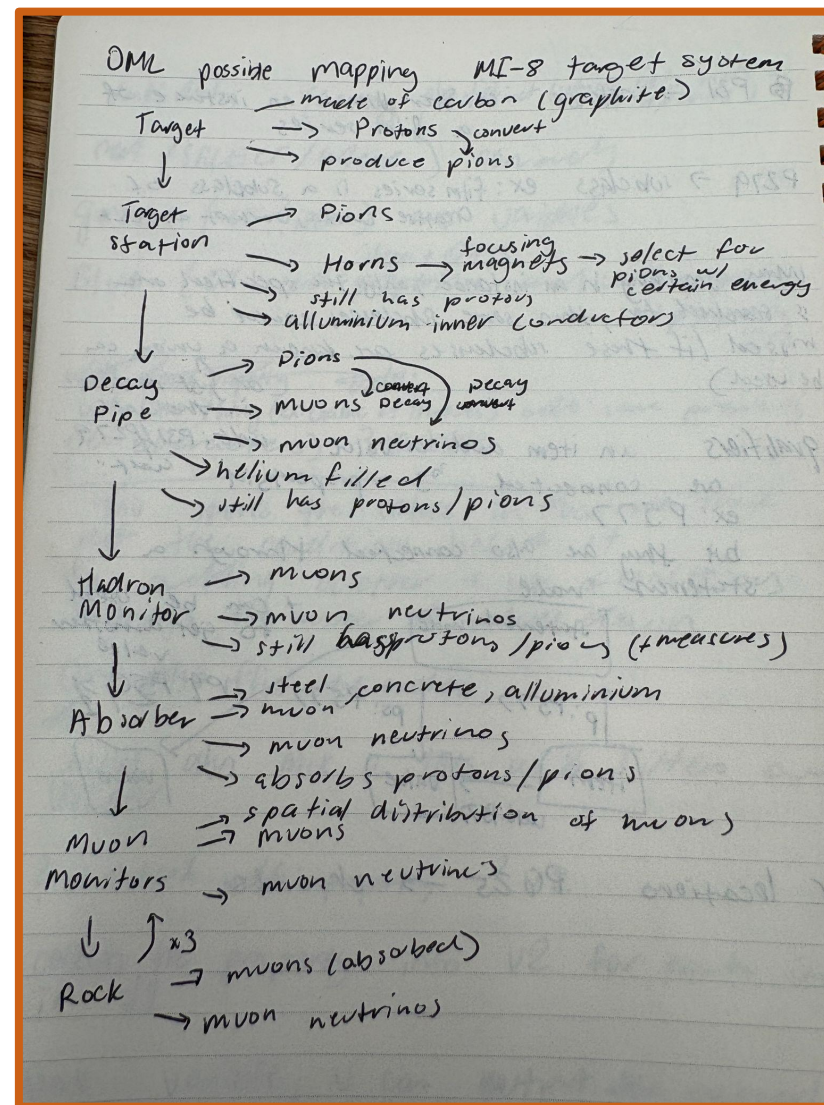
- SPARQL is the coding language used for Wikidata, OML Rosetta, and other similar software
- SPARQL uses groups in the data to effectively pull specific points from its queries

```
1
2 SELECT ?film ?filmLabel WHERE {
3   # Film instance
4   ?film wdt:P31 wd:Q11424 ;
5         # Belongs to the Star Wars series
6         wdt:P179 wd:Q7237422 ;
7   # Human-readable English label
8   SERVICE wikibase:label { bd:serviceParam wikibase:language "[AUTO_LANGUAGE],en". }
9 }
10 |
```



Optimizing OML and SPARQL

- Created a Mind Map for the MI-8 Target. Then, had qwen coder in OpenCode create OML vocabularies/description files grouping the things together
- Have the LLM create specific SPARQL queries





- Sometimes the queries do not get the correct result
- In this case it did not consider that multiple particles were entering/leave sections of the detector (a clear example of the problem that these LLMs face)
- By letting it know its wrong, it changed the query to be correct and get the correct result

Lists all particles/
sections

Optimizing OML and SPARQL part 3

```
C: > MI8TargetAICopy > build > results > {} new-particles.json > {} results > {} bindings > {} 0 > {} newParticle > abc type
1  { "head": {
2    "vars": [ "section" , "newParticle" ]
3  } ,
4  "results": {
5    "bindings": [
6      {
7        "section": { "type": "uri" , "value": "http://example.com/MI8Target/description/sections#target" } ,
8        "newParticle": { "type": "uri" , "value": "http://example.com/MI8Target/description/sections#pion" }
9      } ,
10     {
11       "section": { "type": "uri" , "value": "http://example.com/MI8Target/description/sections#decayPipe" } ,
12       "newParticle": { "type": "uri" , "value": "http://example.com/MI8Target/description/sections#muon" }
13     } ,
14     {
15       "section": { "type": "uri" , "value": "http://example.com/MI8Target/description/sections#decayPipe" } ,
16       "newParticle": { "type": "uri" , "value": "http://example.com/MI8Target/description/sections#muonNeutrino" }
17     }
18   ]
19 }
20 }
21
```

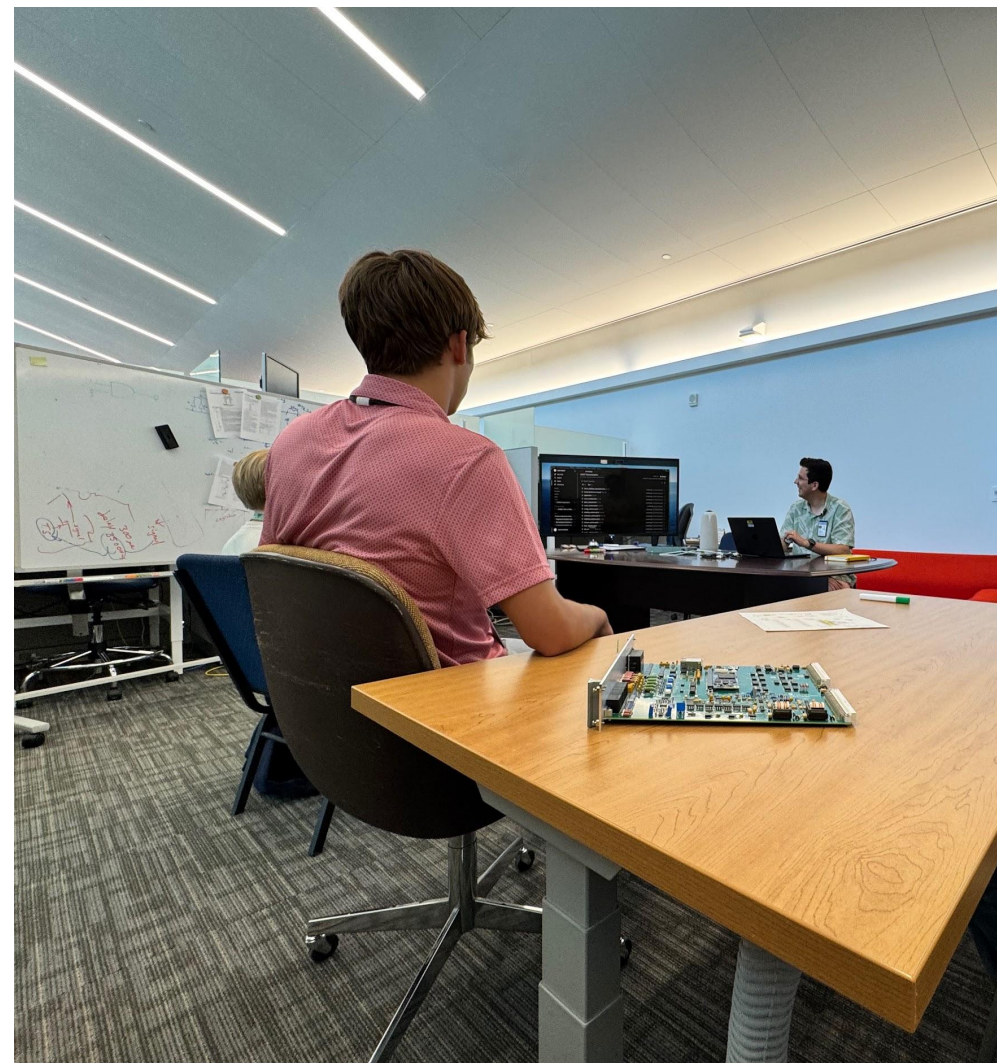
```
> MI8TargetAICopy > src > sparql > new-particles.sparql
1  PREFIX part: <http://example.com/MI8Target/vocabulary/part#>
2  PREFIX sections: <http://example.com/MI8Target/description/sections#>
3
4  SELECT ?section ?newParticle
5  WHERE {
6    ?section a part:Section ;
7    |      | part:producesParticle ?newParticle .
8    FILTER NOT EXISTS {
9      ?section part:receivesParticle ?newParticle .
10    }
11  }
12
```

- Through OpenCode, GPT-OSS edited the sparql query to find the necessary information to answer the question
- The .json results file listed the Pion, Muon, and Muon Neutrinos and their sections they originated from (Target, Decay Pipe, Decay Pipe)
- These results can be easily interpreted by an LLM to directly answer the original question



SPARQL integration through Open WebUI tools

- Since SPARQL queries are a new and complex coding language we wanted the AI to be able to automate them for us
- To solve this problem we used OpenwebUI
- For this I trained the best suited LLM(GPT-OSS) on the tutorial for OML Rosetta and SPARQL queries and gave it access to our OML database





Open WebUI tool system

- To actually run SPARQL queries within Open WebUI I had to create specific tools for the LLM to use
- The first tool translates the user's prompt into SPARQL code, the second tool checks that SPARQL code for errors and edits it if necessary, then the code is shown to the user to check, who if they approve will activate the third tool: which runs the query against the provided OML Rosetta database.

Models Knowledge Prompts Tools



FermiGPT datasearch v0.2

fermigpt-datasearch-v02

Base Model (From)

gpt-oss:120b

Description

Translates and runs sparql queries against an OML Rosetta data set

Add a tag...

Reset Image

Model Params

System Prompt

You are an assistant for querying an OML-Rosetta-derived RDF/OWL knowledge graph via SPARQL.

Workflow, always in this order:

1. Call `get_ontology_schema` (and `search_ontology_terms` for anything not in that summary) before writing any SPARQL. Never invent class, property, or prefix names — use only what these tools return.
2. Draft a SPARQL query using the exact PREFIX lines from `get_ontology_schema`.
3. Call `validate_sparql` on the draft. If it reports an error, fix the query using the schema/search tools and validate again. Do not show an unvalidated query to the user.
4. Once valid, show the user the exact query in a fenced ````sparql` code block, in plain language explain what it will return, and ask them to confirm before running it.
5. Only call `run_sparql_query` after the user has explicitly agreed. If the tool returns `needs_confirmation`, that means no interactive dialog was available — ask in chat and re-call with `confirmed=True` only after an explicit yes.
6. Present results as a table, and note if they were truncated by the row limit. If a query returns zero rows, say so plainly rather than assuming the pattern is wrong.

Never attempt to write, modify, or delete data. If a user asks for an update/insert/delete, explain that this tool is read-only.

Conclusion

Hallucinations are a huge problem that, until stopped, will continue to limit the possibilities of LLMs and other AI tools. Unfortunately it will probably be at least a few years before any significant progress towards this goal is made, and we don't know how long until its completely resolved.



What's next?

While our attempt at solving this problem is still just a rough model, our mentor Javier has introduced us to new tools such as fuzzy search inside Kerberos, that are the future of fighting this issue.



Special
Thanks to:



Javier Contreras

Matteo Piontan Modiano

David Espinoza

Amanda Early

Jose Soto

George Dzuricko



Fermi**FORWARD**



U.S. DEPARTMENT
of ENERGY